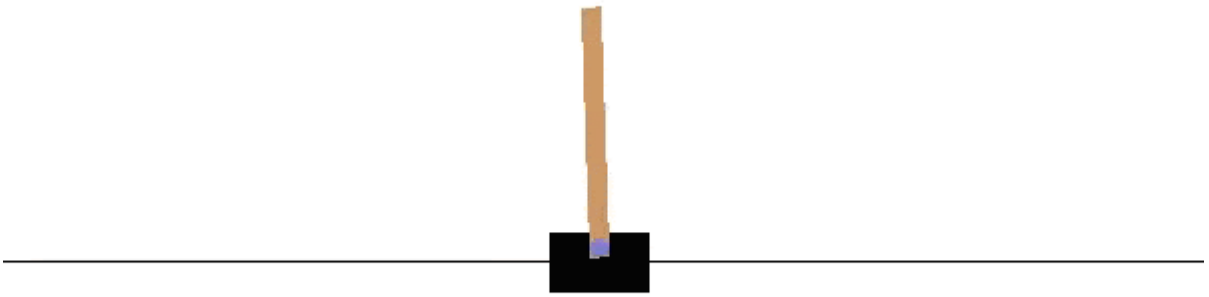
Proyecto Intermodular
IA para CartPole



Realizado por: Maximiliano González Segura

Índice

Arquitectura del Sistema **3**

Modelo de Datos (POO) **5**

Métodos Core **6**

Guía de Despliegue **6**

Pruebas Realizadas **7**

Manual De Usuario **7**

Introducción 7

Primer Uso (Instalación y Arranque) 7

Funcionalidades Principales 8

Modo Entrenamiento 8

Modo Visualización 8

Preguntas Frecuentes (FAQ) 9

Solución de Problemas Comunes 10

Arquitectura del Sistema

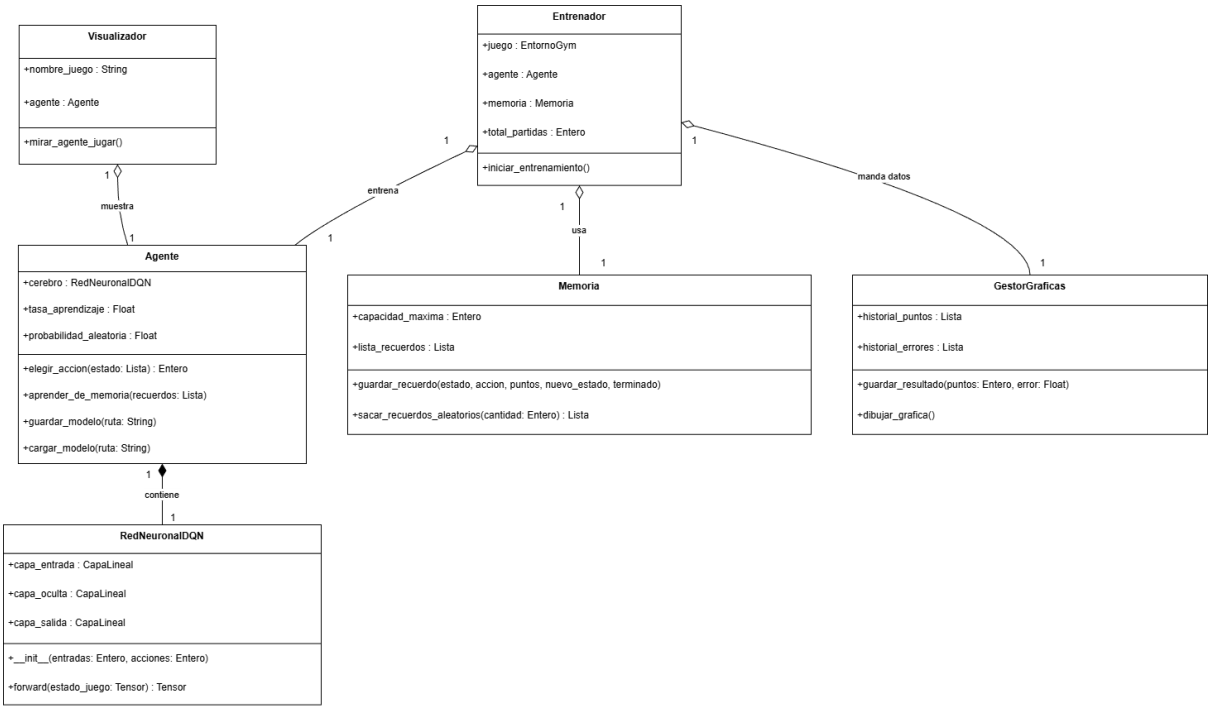
Componentes Principales y Comunicación

El sistema de Inteligencia Artificial se ha modularizado siguiendo el paradigma de Programación Orientada a Objetos (POO). La arquitectura consta de tres pilares:

Entorno (Environment): Proveído por Gymnasium (CartPole-v1), simula la física del juego.

Agente Inteligente: Encapsula la lógica de decisión y contiene la Red Neuronal Profunda (DQN).

Controlador (Entrenador/Visualizador): Gestiona el bucle de tiempo, enviando los estados del entorno al Agente y aplicando las acciones devueltas.



Tecnologías por Componente

Para el Core del Proyecto y de la propia IA, se ha usado **Python 3.12** y **PyTorch 2.2**, para el renderizado y el motor físico se ha usado **Gymnasium** y **pygame-ce**. Para las métricas y análisis se ha usado **Matplotlib**.

Flujo de Datos

Para entender cómo funciona el proyecto por dentro, podemos ver el flujo de datos como una conversación constante entre el videojuego (el Entorno) y nuestro jugador ciego (el Agente). Este ciclo se repite en bucle y tiene 5 pasos muy claros:

1. Lectura de la situación (Estado): Como nuestra IA no tiene ojos para ver la pantalla, el juego empieza mandándole una "foto" de la situación actual. Esta foto es simplemente un array de 4 números: dónde está el carro, a qué velocidad va, cuánto está inclinado el palo y a qué velocidad se está cayendo.

2. Traducción y Decisión: Nuestra IA recibe esos 4 números, pero su cerebro solo entiende un formato especial llamado "Tensor". Así que convertimos esos números a Tensor. En este punto, la IA decide qué hacer: a veces pulsa un botón al azar para explorar cómo funciona el juego, y otras veces usa su red neuronal para intentar ganar (política llamada Epsilon-Greedy). Al final, elige una acción: Izquierda (0) o Derecha (1).

3. Acción y Respuesta (Step): Le mandamos esa acción al juego. El juego mueve el carro, avanza un fotograma de tiempo y nos devuelve los resultados: nos da un punto de recompensa si el palo sigue de pie, nos avisa si hemos perdido, y nos manda los 4 números nuevos con la posición actualizada.

4. Guardado en Memoria (Replay Buffer): Para que la IA pueda aprender, necesita recordar sus errores. Por eso, cogemos todo lo que acaba de pasar (qué botón pulsamos, qué puntos ganamos…) y lo guardamos en un cajón o historial que llamamos **Memoria**.

5. Aprendizaje y Corrección: La IA no se hace inteligente al instante. De vez en cuando, nuestro programa saca un puñado de recuerdos aleatorios de esa Memoria y los estudia. Si en un recuerdo ve que mover a la izquierda hizo que el palo se cayera, calcula el error y ajusta un poco el "cerebro" matemático de la red neuronal para que no vuelva a cometer ese mismo fallo en el futuro.

Modelo de Datos (POO)

Dado que la naturaleza del sistema no requiere una Base de Datos relacional (SQL) para su funcionamiento en tiempo real, el modelo de datos se implementa en estructuras de memoria RAM*:*

Estructura Replay Buffer (Memoria): Colección tipo "deque" con límite de capacidad (ej. 10.000 registros). Almacena experiencias pasadas para evitar el "olvido catastrófico" de la red neuronal.

Tensores PyTorch (DQN): Estructuras matemáticas multidimensionales que almacenan los pesos y sesgos de las capas ocultas (Linear layers) del cerebro artificial.

Métodos Core

Al no disponer de un backend web tradicional con endpoints HTTP, las interfaces de comunicación del sistema se basan en métodos de clase:

Entidad	Método	Descripción
Agente	elegir_accion(estado_tensor)	Recibe el estado actual y devuelve un entero (0 o 1) con la decisión a tomar.
Entrenador	aprender_de_memoria()	Extrae un lote aleatorio de la Memoria, aplica la ecuación de Bellman (Loss) y ajusta los pesos del optimizador.
Gymnasium	env.step(action)	Procesa la acción elegida y avanza la física de la simulación.

Guía de Despliegue

Requisitos del sistema: Windows 10/11, macOS o Linux con entorno de ventanas (X11) habilitado y Python 3.10+.

Pasos de instalación:

1. Clonar el repositorio mediante Git.
2. Activar un entorno virtual (recomendado): **python -m venv venv**
3. Ejecutar: **pip install -r requirements.txt** (o instalar dependencias manualmente: **torch gymnasium pygame-ce matplotlib**).

Variables y Bases de Datos: El sistema a nivel de infraestructura guarda su conocimiento localmente en un archivo binario **.pth**, por lo que no requiere configuración de BBDD

Pruebas Realizadas

Se ha implementado testing técnico mediante la librería nativa **unittest**.

Las pruebas **cubren el 100% de los módulos críticos**: inicialización de variables POO en el Agente, respeto de los límites del Replay Buffer en la Memoria, dimensiones de salida correctas del modelo DQN y carga de instancias en el Entrenador.

Durante el desarrollo se detectó una velocidad incontrolable de renderizado ("Headless execution"). Se solucionó gestionando framerates con **time.sleep().**

Manual De Usuario

Introducción

En este breve manual os explico cómo utilizar el módulo de Inteligencia Artificial (IA) que he desarrollado para el Proyecto Intermodular. Mi objetivo con esta aplicación es que podáis comprobar, de forma muy visual y práctica,

cómo el Agente Inteligente que he programado aprende desde cero a dominar el entorno CartPole (mantener un péndulo en equilibrio sobre un carro).

He diseñado este software para que cualquier persona pueda iniciar las simulaciones de aprendizaje profundo y ver los resultados en pantalla sin necesidad de tocar el código ni tener conocimientos matemáticos avanzados.

Primer Uso (Instalación y Arranque)

Como este módulo es un sistema de Inteligencia Artificial pensado para ejecutarse de forma local, no he implementado un sistema de registro de usuarios ni inicio de sesión con contraseñas.

Para que podáis probar el programa en vuestro equipo por primera vez, solo necesitáis tener Python instalado. Abrid una terminal en la carpeta del proyecto y ejecutad este comando para instalar las librerías que he utilizado para la red neuronal y los gráficos:

pip install torch gymnasium[classic_control] pygame-ce matplotlib

Una vez termine de instalarse, mi sistema estará listo para funcionar.

Funcionalidades Principales

He dividido la ejecución del proyecto en dos modos principales según lo que queráis probar. Aquí os explico cómo usar cada uno:

Modo Entrenamiento

Si queréis poner a la IA a jugar miles de partidas para que aprenda de sus errores, abrid la consola en la raíz del proyecto y ejecutad mi script de entrenamiento:

python src/entrenador.py

Veréis que la consola empieza a mostrar en tiempo real cómo avanza mi agente (qué episodio está jugando, cuántos puntos saca y cómo va reduciendo su nivel de movimientos locos/aleatorios). Al terminar, mi código genera automáticamente un archivo "cerebro" (.pth) en la carpeta models/ y dibuja una gráfica de aprendizaje.

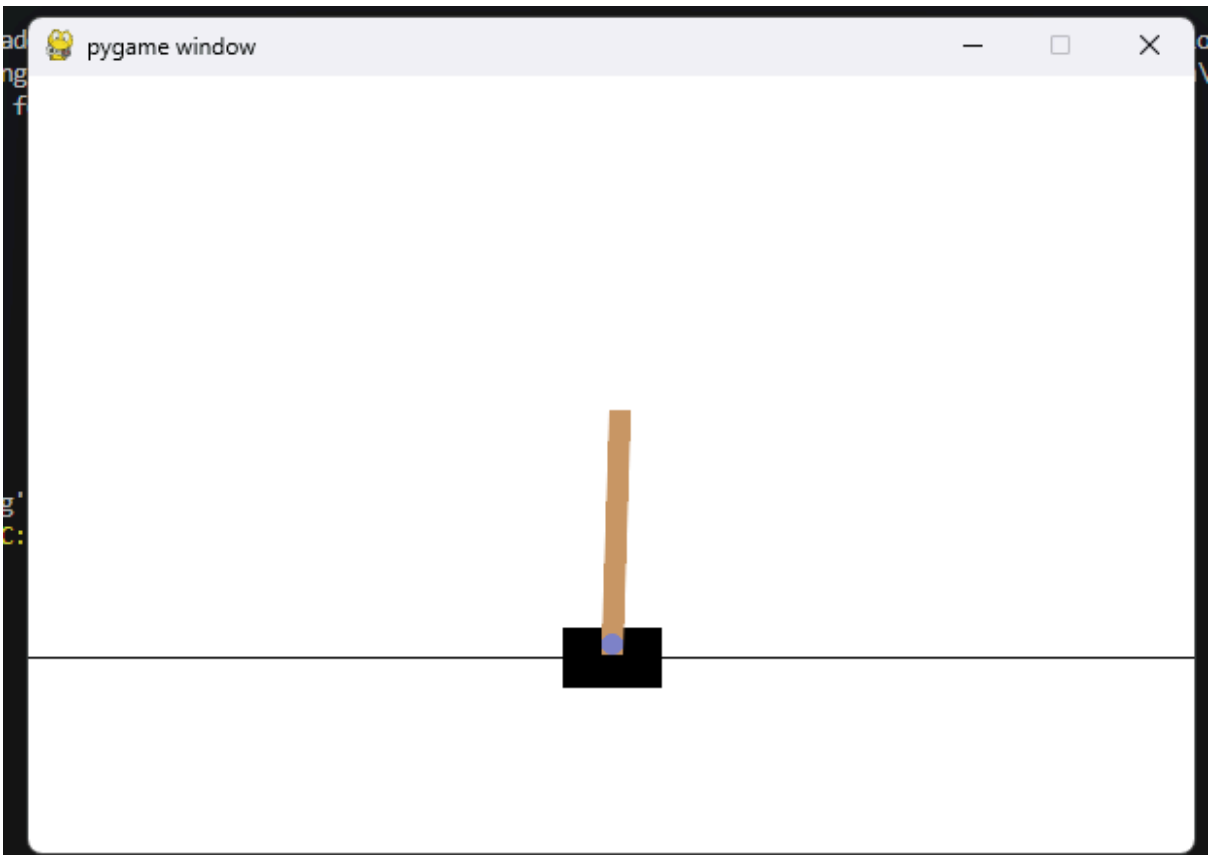
```
py.ndarray with numpy.array() before converting to a tensor. (Triggered internally at C:\actions-runner\_work\pytorch\pytorch\pytorch\torch\csrc\utils\tensor_new.cpp:256.)
  estados = torch.FloatTensor([experiencia[0] for experiencia in lote])
Episodio 10/100 | Puntos: 34 0.95
Episodio 20/100 | Puntos: 12 0.90
Episodio 30/100 | Puntos: 11 0.86
Episodio 40/100 | Puntos: 24 0.82
Episodio 50/100 | Puntos: 52 0.78
Episodio 60/100 | Puntos: 26 0.74
Episodio 70/100 | Puntos: 26 0.70
Episodio 80/100 | Puntos: 143 0.67
Episodio 90/100 | Puntos: 72 0.64
Episodio 100/100 | Puntos: 123 0.61
¡Entrenamiento finalizado!
Gráfica guardada como 'grafica_aprendizaje.png'
PS C:\Users\Maxi\Desktop\2DO-DAM\PROYECTO>
```

Modo Visualización

Si lo que queréis es ver el resultado final de forma gráfica, simulando el juego a una velocidad normal, tenéis que ejecutar:

python src/visualizacion.py

El programa abrirá una ventana donde podréis ver físicamente al carro intentando balancear el péndulo, reaccionando en milisegundos a las decisiones que toma la red neuronal.



Preguntas Frecuentes (FAQ)

¿Por qué al abrir la visualización el péndulo se cae a los pocos segundos?

Si ejecutáis el juego sin entrenar a la IA primero, el agente empieza en fase de exploración pura. Es decir, lo he programado para que al principio pulse botones 100% al azar para descubrir cómo funciona la gravedad del juego, por lo que es normal que pierda muy rápido.

¿Dónde guarda mi programa los resultados tras entrenar a la IA? He configurado el sistema para que genere dos archivos clave en vuestras carpetas:

> **grafica_aprendizaje.png** (en la carpeta assets/): Una imagen que dibuja la curva de lo que ha aprendido la IA.
>
> **cerebro_cartpole.pth** (en la carpeta models/): Es el archivo que guarda los "pesos" y la memoria de la red neuronal.

Solución de Problemas Comunes

Si al probar mi proyecto os topáis con algún error, aquí detallo los más comunes y cómo los podéis solucionar:

- **Error: ModuleNotFoundError: No module named 'gymnasium'**
  - **Causa:** Vuestro entorno de Python no detecta las librerías del juego.
  - **Solución:** Aseguraos de haber ejecutado el comando pip install que menciono en el apartado de Primer Uso.
- **La ventana del juego no se abre o crashea de golpe ("Headless Error")**
  - **Causa:** Estáis intentando ejecutar la visualización en una terminal sin acceso a interfaz gráfica (como servidores en la nube o WSL en Windows).
  - **Solución:** Ejecutad mi script visualizacion.py desde una consola normal de vuestro sistema operativo (CMD o PowerShell en Windows).
- **El entrenamiento va demasiado lento:**
  - **Solución:** El entrenamiento matemático tira bastante del procesador. Os recomiendo cerrar otros programas pesados mientras la IA hace sus cálculos.